Computer Architecture
Arithmetic for Computers

Tatpong Katanyukul

CoE, KKU

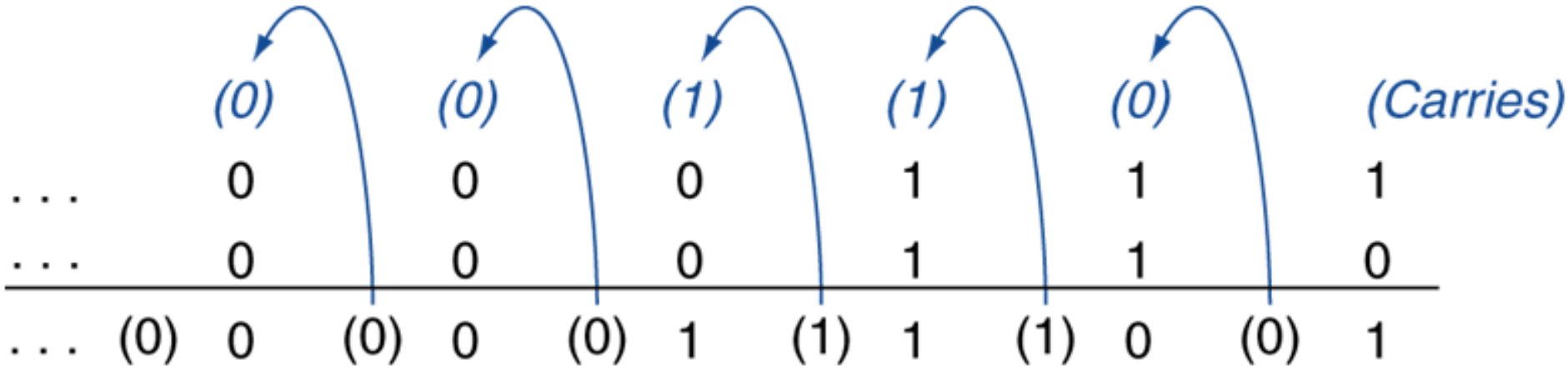
24 พฤษภาคม พ.ศ. 2561

Arithmetic for Computers

- Operations on integers
  - Addition and subtraction
  - Multiplication and division
  - Dealing with overflow
- Floating-point real numbers
  - Representation and operations

Integer Addition

Example: 7 + 6



Four-Bit Words and Representative Capacities

$[0111] \Leftrightarrow 7$	$[1111] \Leftrightarrow -1$
$[0110] \Leftrightarrow 6$	$[1110] \Leftrightarrow -2$
$[0101] \Leftrightarrow 5$	$[1101] \Leftrightarrow -3$
$[0100] \Leftrightarrow 4$	$[1100] \Leftrightarrow -4$
$[0011] \Leftrightarrow 3$	$[1011] \Leftrightarrow -5$
$[0010] \Leftrightarrow 2$	$[1010] \Leftrightarrow -6$
$[0001] \Leftrightarrow 1$	$[1001] \Leftrightarrow -7$
$[0000] \Leftrightarrow 0$	$[1000] \Leftrightarrow -8$

Word Bits and Representative Capacities

4-bit words

$[0111]_2 \Leftrightarrow 7$

$\vdots \quad \vdots \quad \vdots \quad \vdots$

$[0000]_2 \Leftrightarrow 0$

$\vdots \quad \vdots \quad \vdots \quad \vdots$

$[1000]_2 \Leftrightarrow -8$

8-bit words

$[01111111]_2 \Leftrightarrow 127$

$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$

$[00000010]_2 \Leftrightarrow 2$

$[00000001]_2 \Leftrightarrow 1$

$[00000000]_2 \Leftrightarrow 0$

$[11111111]_2 \Leftrightarrow -1$

$[11111110]_2 \Leftrightarrow -2$

$[11111101]_2 \Leftrightarrow -3$

$[11111100]_2 \Leftrightarrow -4$

$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$

$[10000000]_2 \Leftrightarrow -128$

32-bit words

- $\max = 2,147,483,647$
- $\min = -2,147,483,648$

Four-Bit Additions

3 + 2	3 + -2	-3 + 2	-3 + -2
[0011] +	[0011] +	[1101] +	[1101] +
[0010]	[1110]	[0010]	[1110]
-------	-------	-------	-------
-------	-------	-------	-------

$[0111] \Leftrightarrow 7$
$[0110] \Leftrightarrow 6$
$[0101] \Leftrightarrow 5$
$[0100] \Leftrightarrow 4$
$[0011] \Leftrightarrow 3$
$[0010] \Leftrightarrow 2$
$[0001] \Leftrightarrow 1$
$[0000] \Leftrightarrow 0$
$[1111] \Leftrightarrow -1$
$[1110] \Leftrightarrow -2$
$[1101] \Leftrightarrow -3$
$[1100] \Leftrightarrow -4$
$[1011] \Leftrightarrow -5$
$[1010] \Leftrightarrow -6$
$[1001] \Leftrightarrow -7$
$[1000] \Leftrightarrow -8$

Four-Bit Additions

3 + 2	3 + -2	-3 + 2	-3 + -2
[0011] +	[0011] +	[1101] +	[1101] +
[0010]	[1110]	[0010]	[1110]
--------	--------	--------	--------
[0101] = 5	[0001] = 1	[1111] = -1	[1011] = -5
-------	-------	-------	-------

$[0111] \Leftrightarrow 7$
$[0110] \Leftrightarrow 6$
$[0101] \Leftrightarrow 5$
$[0100] \Leftrightarrow 4$
$[0011] \Leftrightarrow 3$
$[0010] \Leftrightarrow 2$
$[0001] \Leftrightarrow 1$
$[0000] \Leftrightarrow 0$
$[1111] \Leftrightarrow -1$
$[1110] \Leftrightarrow -2$
$[1101] \Leftrightarrow -3$
$[1100] \Leftrightarrow -4$
$[1011] \Leftrightarrow -5$
$[1010] \Leftrightarrow -6$
$[1001] \Leftrightarrow -7$
$[1000] \Leftrightarrow -8$

Four-Bit Additions

4 + 5

[ ] +
[ ]

[ ]
---

4 + -5

[ ] +
[ ]

[ ]

-4 + 5

[ ] +
[ ]

[ ]

-4 + -5

[ ] +
[ ]

[ ]

$[0111] \Leftrightarrow 7$
$[0110] \Leftrightarrow 6$
$[0101] \Leftrightarrow 5$
$[0100] \Leftrightarrow 4$
$[0011] \Leftrightarrow 3$
$[0010] \Leftrightarrow 2$
$[0001] \Leftrightarrow 1$
$[0000] \Leftrightarrow 0$
$[1111] \Leftrightarrow -1$
$[1110] \Leftrightarrow -2$
$[1101] \Leftrightarrow -3$
$[1100] \Leftrightarrow -4$
$[1011] \Leftrightarrow -5$
$[1010] \Leftrightarrow -6$
$[1001] \Leftrightarrow -7$
$[1000] \Leftrightarrow -8$

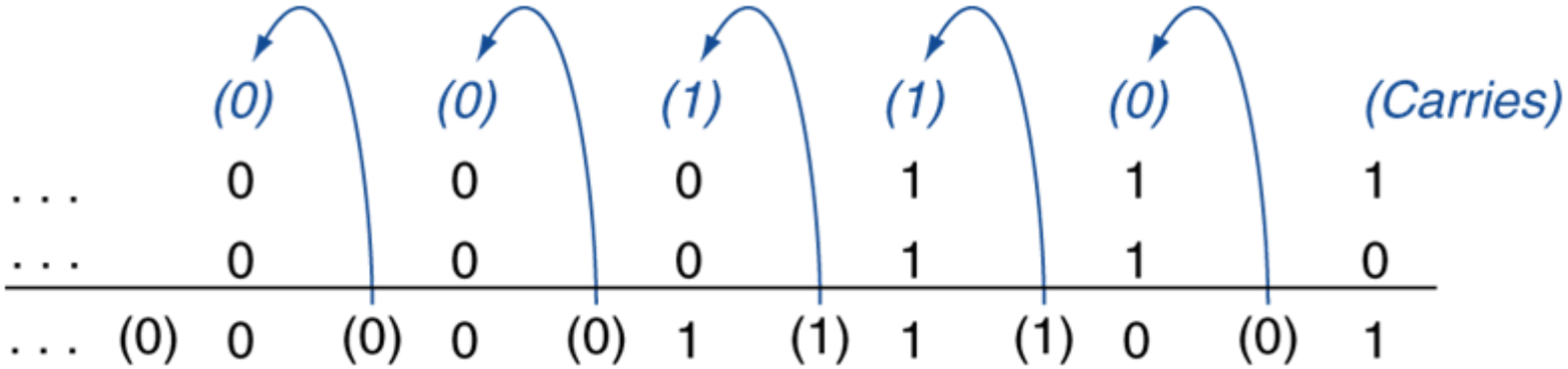
Four-Bit Additions

4 + 5	4 + -5	-4 + 5	-4 + -5
[0100] +	[0100] +	[1100] +	[1100] +
[0101]	[1011]	[0101]	[1011]
--------	--------	--------	--------
[1001] = -7 (!?!)	[1111] = -1	[0001] = 1	[0111] = 7 (!?!)
--------	--------	--------	--------

$[0111] \Leftrightarrow 7$
$[0110] \Leftrightarrow 6$
$[0101] \Leftrightarrow 5$
$[0100] \Leftrightarrow 4$
$[0011] \Leftrightarrow 3$
$[0010] \Leftrightarrow 2$
$[0001] \Leftrightarrow 1$
$[0000] \Leftrightarrow 0$
$[1111] \Leftrightarrow -1$
$[1110] \Leftrightarrow -2$
$[1101] \Leftrightarrow -3$
$[1100] \Leftrightarrow -4$
$[1011] \Leftrightarrow -5$
$[1010] \Leftrightarrow -6$
$[1001] \Leftrightarrow -7$
$[1000] \Leftrightarrow -8$

Integer Addition

Example: 7 + 6



Overflow if result out of range

- Adding +value and value operands, no overflow
- Adding two +value operands

  Overflow if result sign is 1 ($\equiv$ −value).
- Adding two −value operands

  Overflow if result sign is 0 ($\equiv$ +value).

Integer Subtraction

- Add negation of second operand
- Example: $7 - 6 = 7 + (-6)$

  +7: 0000 0000 ... 0000 0111

  -6: 1111 1111 ... 1111 1010

  +1: 0000 0000 ... 0000 0001
- Overflow if result out of range
  - Subtracting two +value or two value operands, no overflow
  - Subtracting +value from −value operand, e.g., $-A - (+B)$
    Overflow if result sign is 0
  - Subtracting −value from +value operand, e.g., $+A - (-B)$
    Overflow if result sign is 1

Dealing with Overflow

- Some languages (e.g., C) ignore overflow
  - Use MIPS addu, addui, subu instructions
- Other languages (e.g., Ada, Fortran) require raising an exception
  - Use MIPS add, addi, sub instructions
  - On overflow, invoke exception handler
    - Save PC in exception program counter (EPC) register
    - Jump to predefined handler address
    - Once exception handling is done, mfc0 (move from coprocessor reg) instruction can retrieve EPC value, to return after corrective action
    - <We'll see an example shortly.>

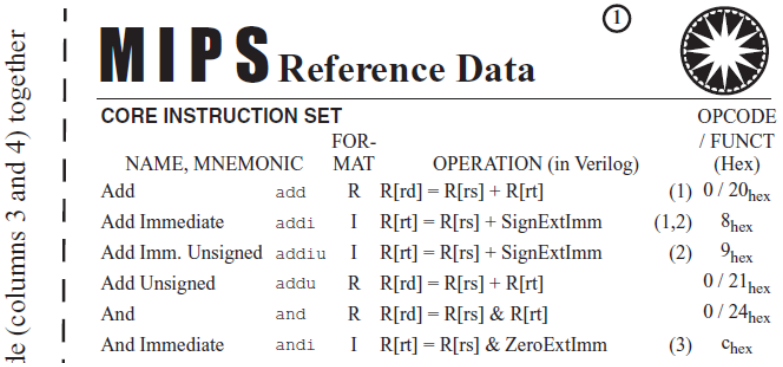
MIPS: Overflow Exception

```
        .globl main
        .text
main:
addi   $s0, $0, 20
addiu  $s0, $0, 20

addi   $s0, $0, -20
addiu  $s0, $0, -20

li     $s0, 0x7FFFFFFF
addiu  $s1, $s0, 1
addi   $s1, $s0, 1
nop
```

- Check what addiu and addi do.



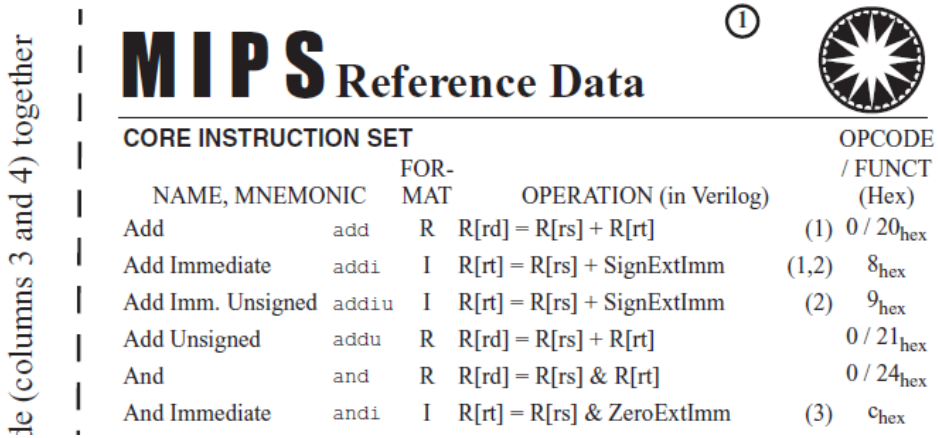
le (columns 3 and 4) together

MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0 / 20_{hex}$
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0 / 21_{hex}$
And	and	R	R[rd] = R[rs] & R[rt]		$0 / 24_{hex}$
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}

Overflow Exception

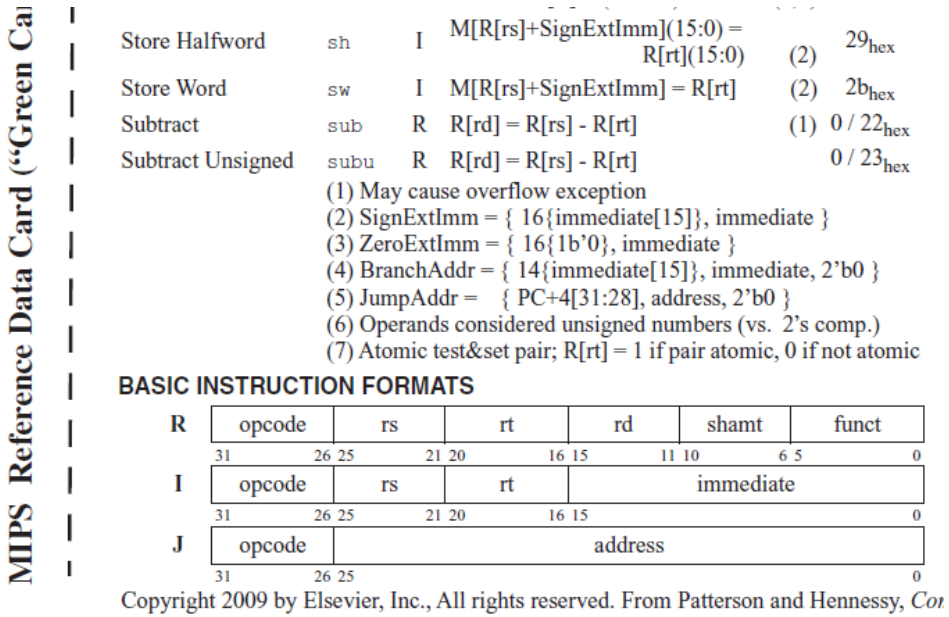


le (columns 3 and 4) together

MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	$0 / 20_{hex}$
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		$0 / 21_{hex}$
And	and	R	R[rd] = R[rs] & R[rt]		$0 / 24_{hex}$
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}



MIPS Reference Data Card ("Green Ca

Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2)	$2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1)	$0 / 22_{hex}$
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]		$0 / 23_{hex}$

(1) May cause overflow exception
(2) SignExtImm = { 16{immediate[15]}, immediate }
(3) ZeroExtImm = { 16{1b'0}, immediate }
(4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }
(5) JumpAddr = { PC+4[31:28], address, 2'b0 }
(6) Operands considered unsigned numbers (vs. 2's comp.)
(7) Atomic test&set pair; R[rt] = 1 if pair atomic, 0 if not atomic

BASIC INSTRUCTION FORMATS

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, *Co*

- Both addiu and addi work with sign extension: see note (2).
- But, only addi can cause overflow exception.

Exception and Interrupt

- Exception: unscheduled event that disrupts program execution
- Interrupt: an exception that comes from outside of the processor. Some architectures use the term interrupt for all exceptions.

Java Exception Example

```
import java.lang.*;
public class jtryCatchExp {
  public static void main( String args[] ){
    int x; int y; int z;
    if( args.length != 2){
      x = 32; y = 12;
    } else {
      x = Integer.parseInt(args[0]); y = Integer.parseInt(args[1]);
    }
    System.out.println("x = " + x + "\ny = " + y);
    try{
      z = x / y;  System.out.println("x / y = " + z);
    } catch(Exception e){
      System.out.println("catch it");
    }
    System.out.println("Do something else and done.");
  }//main
}//class
```

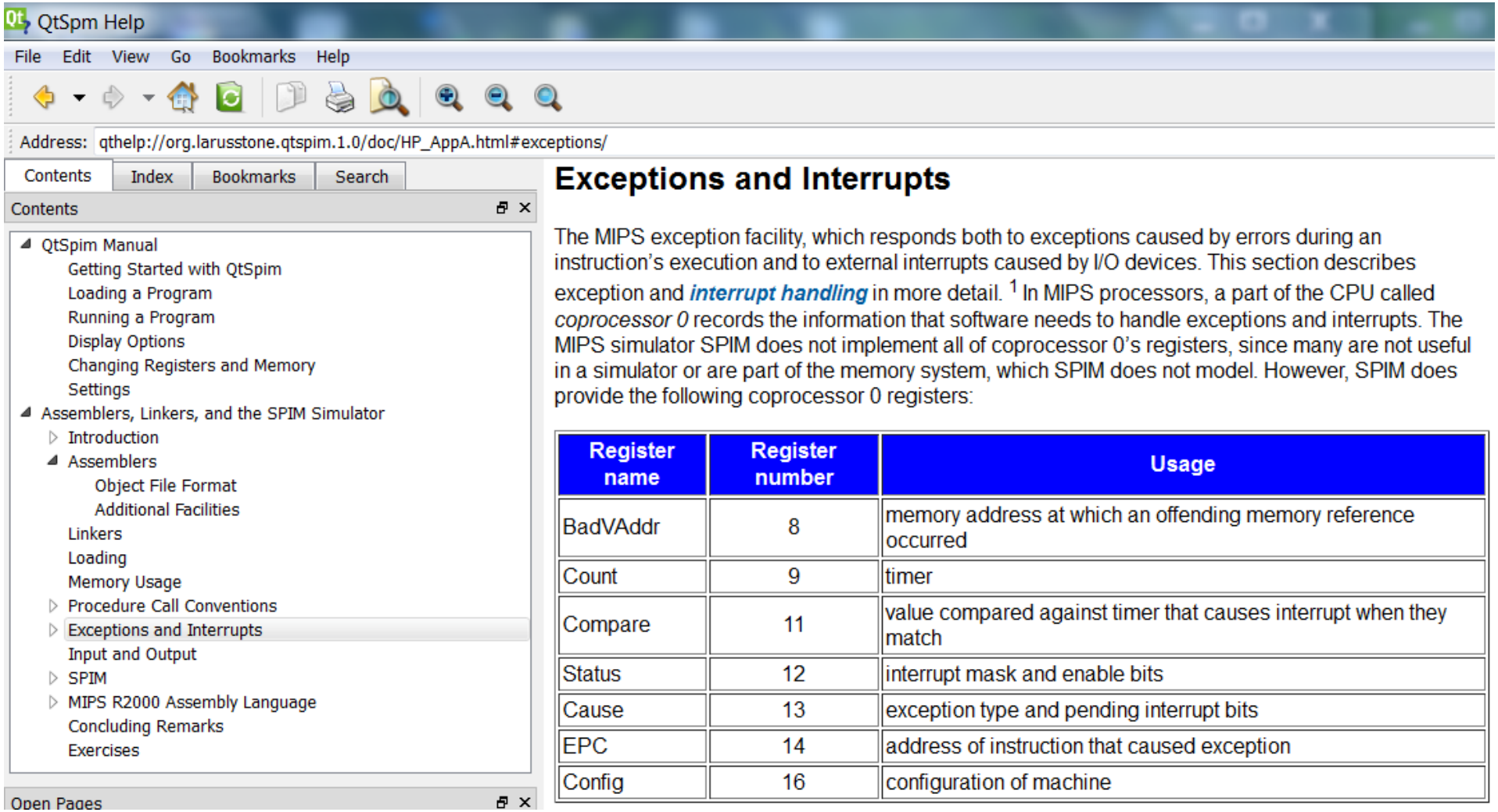
Confer to Java Exception Example Without Try-Catch

```
import java.lang.*;
public class jtryCatchExp {
  public static void main( String args[] ){
    int x; int y; int z;
    if( args.length != 2){
      x = 32; y = 12;
    } else {
      x = Integer.parseInt(args[0]); y = Integer.parseInt(args[1]);
    }
    System.out.println("x = " + x + "\ny = " + y);
    z = x / y;  System.out.println("x / y = " + z);
    System.out.println("Do something else and done.");
  }//main
}//class
```

Exception Handling

- Two approaches to identify cause of exception
  - (1) Using multiple handler addresses:
    handler address identifies the cause.
  - (2) Using single handler address with Cause register.
- MIPS takes Cause register approach.

MIPS Exception Handling: Exception-Related Registers

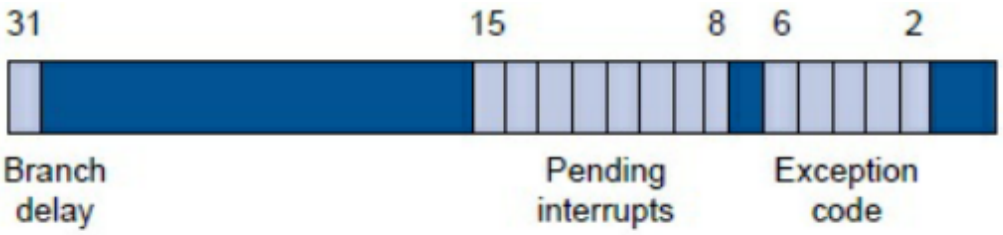


Exceptions and Interrupts

The MIPS exception facility, which responds both to exceptions caused by errors during an instruction's execution and to external interrupts caused by I/O devices. This section describes exception and ***interrupt handling*** in more detail. [1] In MIPS processors, a part of the CPU called *coprocessor 0* records the information that software needs to handle exceptions and interrupts. The MIPS simulator SPIM does not implement all of coprocessor 0's registers, since many are not useful in a simulator or are part of the memory system, which SPIM does not model. However, SPIM does provide the following coprocessor 0 registers:

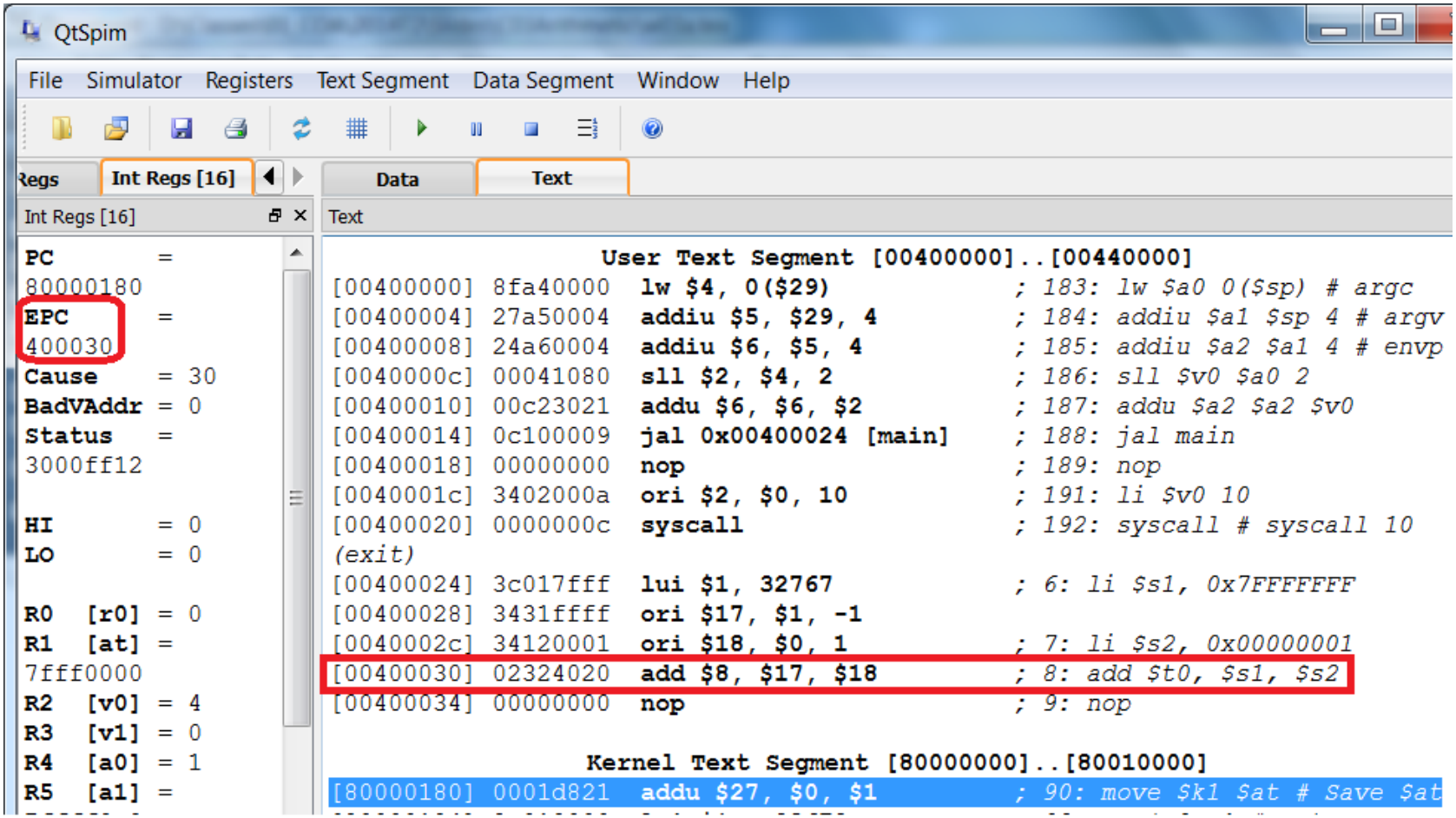
Register name	Register number	Usage
BadVAddr	8	memory address at which an offending memory reference occurred
Count	9	timer
Compare	11	value compared against timer that causes interrupt when they match
Status	12	interrupt mask and enable bits
Cause	13	exception type and pending interrupt bits
EPC	14	address of instruction that caused exception
Config	16	configuration of machine

MIPS Exception Handling: Cause Register

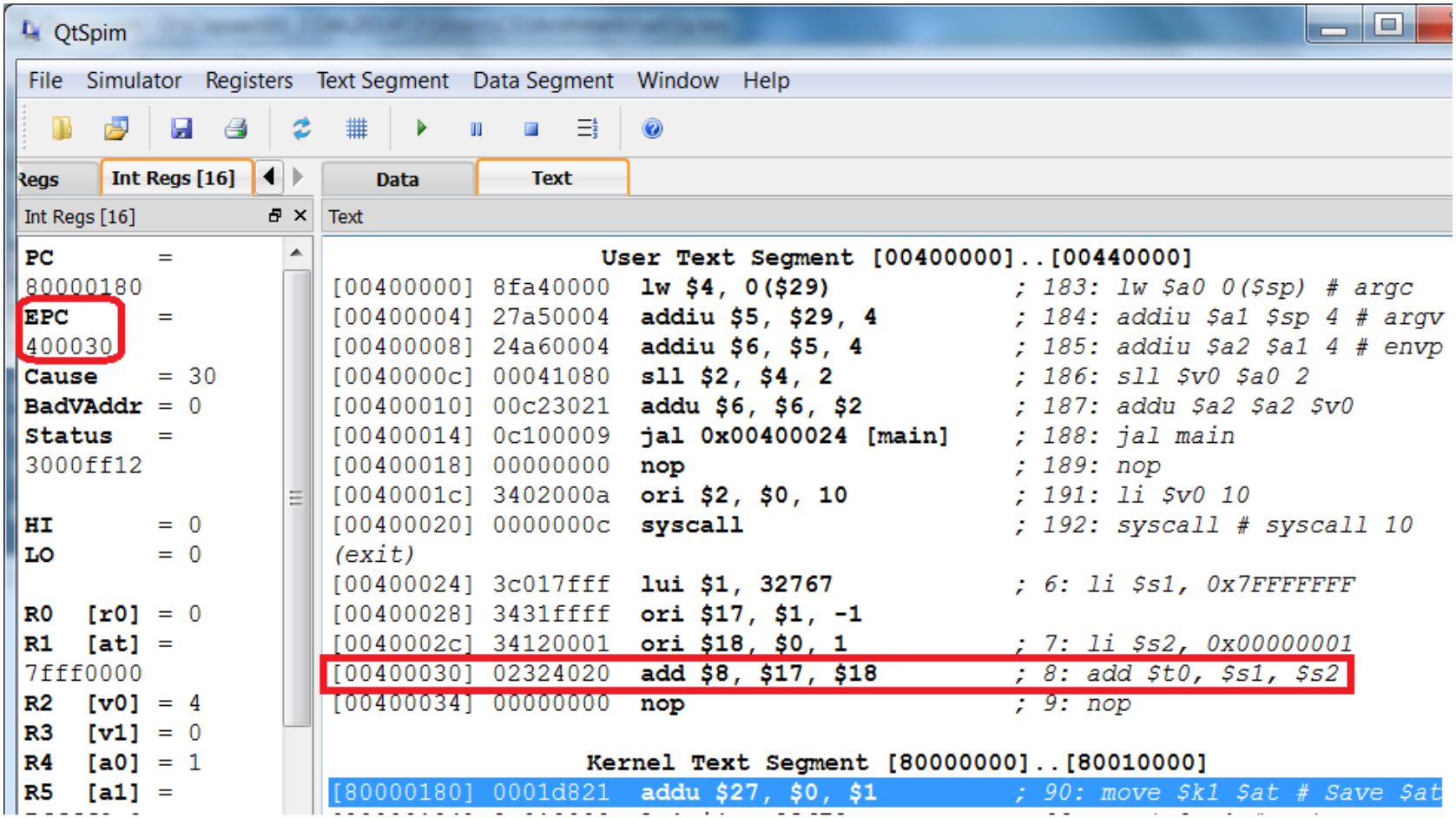


Number	Name	Cause of exception
0	Int	interrupt (hardware)
4	AdEL	address error exception (load or instruction fetch)
5	AdES	address error exception (store)
6	IBE	bus error on instruction fetch
7	DBE	bus error on data load or store
8	Sys	syscall exception
9	Bp	breakpoint exception
10	RI	reserved instruction exception
11	CpU	coprocessor unimplemented
12	Ov	arithmetic overflow exception
13	Tr	trap
15	FPE	floating point

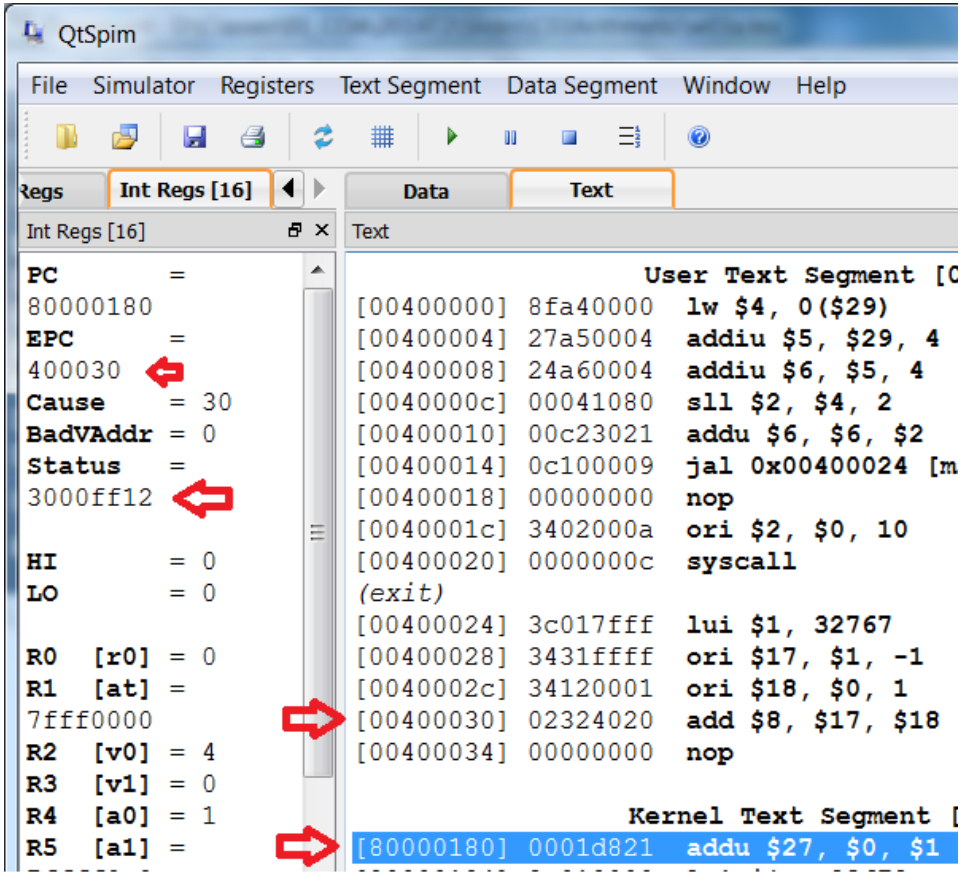
Example: Catching Exception



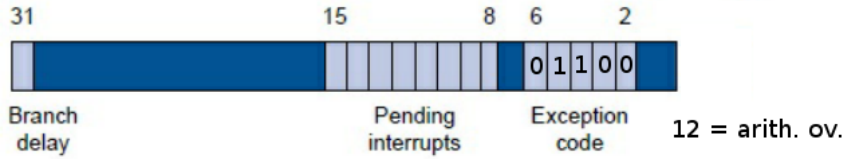
Catch Exception



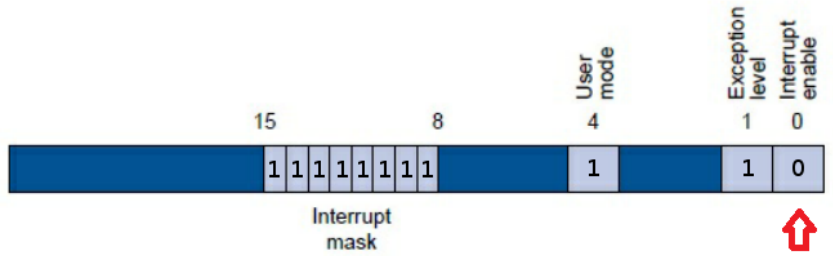
MIPS: Catch Exception



- Instruction caused exception is saved in EPC
- Cause of exception is recored in Cause

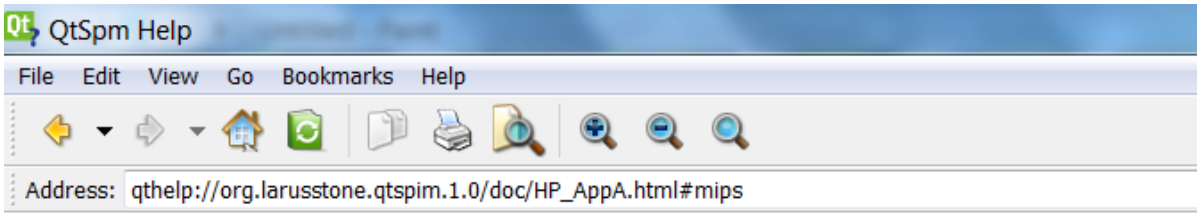


- Diable interrupt, via Status



- PC is assigned to 0x80000180

MIPS: Exception Handling



Exception and Interrupt Instructions

Exception return

eret				
0x10	1	0	0x18	value
6	1	19	6	width (bits)

Set the EXL bit in coprocessor 0's Status register to 0 and return to the instruction pointed to by coprocessor 0's EPC register.

Exception Handling Example

adapted from QtSPIM help: exception

```
1  ## Example Exception Handling: ExpHand01.asm
2  ## We will take saturation approach
3
4  .ktext 0x80000180
5  ## Exception Handling Code
6  ## if $s1 < 0, $s1 must be +sat: 0x7FFFFFFF.
7  ## if $s1 > 0, $s1 must be -sat: 0x80000000.
8
9    slt $t0, $s1, $zero
10   beq $t0, $zero, NegSat
11     li $s1, 0x7FFFFFFF    # PosSat
12     j EHDone
13   NegSat:   li $s1, 0x80000000    # NegSat
14
15 EHDone:
16 mfc0 $k0, $14      # $k0 = EPC  # EHDone
17 addiu $k0, $k0, 4 # Do not reexecute faulting instruction
18 mtc0 $k0, $14      # Update EPC
19 mtc0 $zero, $13    # Clear Cause register
20 mfc0 $k0, $12      # Fix Status register
21 andi $k0, 0xfffd   # Clear Exception Level (EXL bit)
22 ori $k0, 0x1       # Enable interrupts
23 mtc0 $k0, $12      # Update Status register
24 eret               # Return to EPC
25
```

```
26     .globl main
27     .text
28 main:
29 li $s2, 0x10000000
30 li $s0, 0x7FFFFFFF
31 addiu $s1, $s0, 1
32 sw $s1, 0($s2)
33 addi  $s1, $s0, 1
34 sw $s1, 4($s2)
35 nop
```

- Address 0x10000000 has a result of addiu
- Address 0x10000004 has a result of addi
  Our exception handling will turn the result to saturated value.

Move data between general registers and special registers

- interrupt related registers EPC, Status, and Cause, called "co-processor" registers
- co-processor registers require privilege access
- mfc0 (move from control reg.), mtc0 (move to control reg.)

Move from coprocessor 0

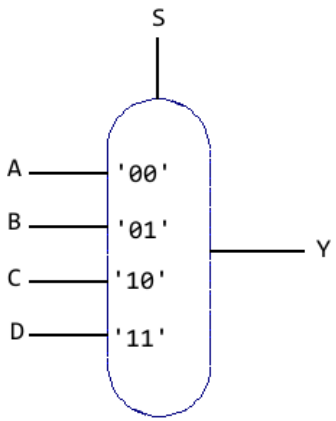
mfc0 rt, rd						
0x10	0	rt	rd	0	0	value
6	5	5	5	5	6	width (bits)

Move to coprocessor 0

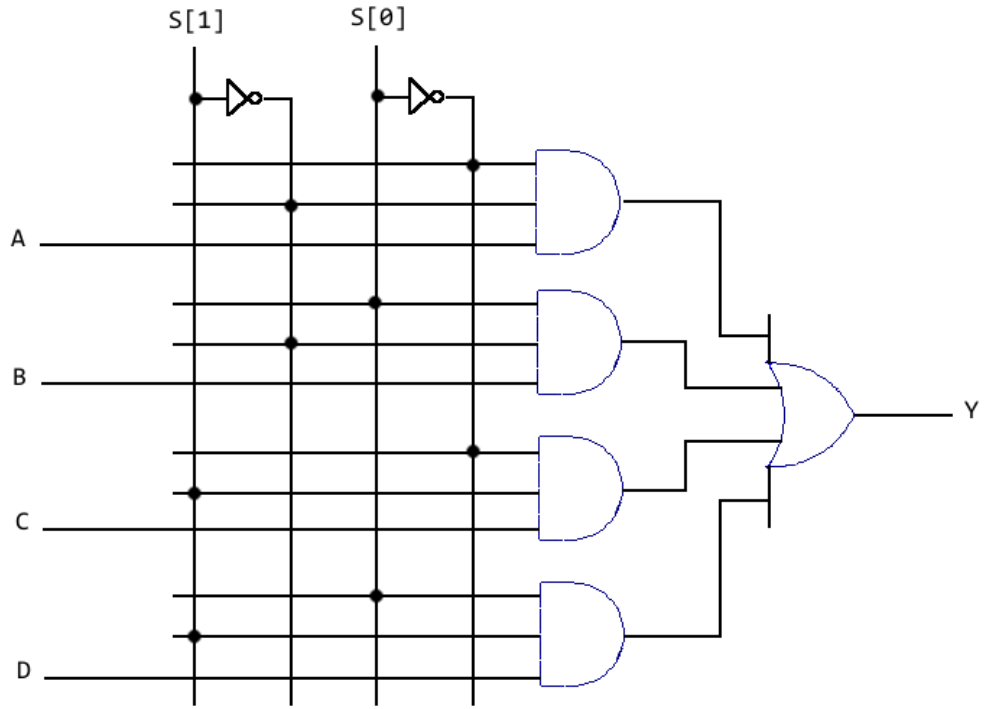
mtc0 rd, rt						
0x10	4	rt	rd	0	0	value
6	5	5	5	5	6	width (bits)

Arithmetic Logic Unit: Multiplexer

- Implementing circuits for addition and subtraction, as well as logical operations, e.g. AND, OR.
- Use a multiplexer to selection an appropriate circuit.

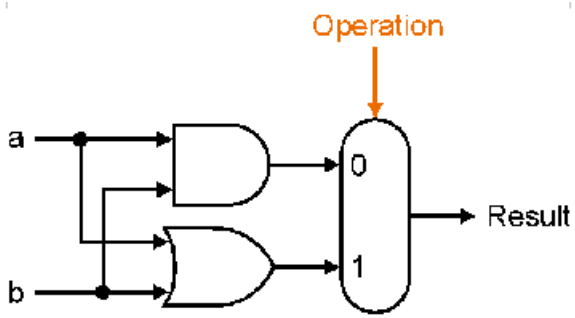


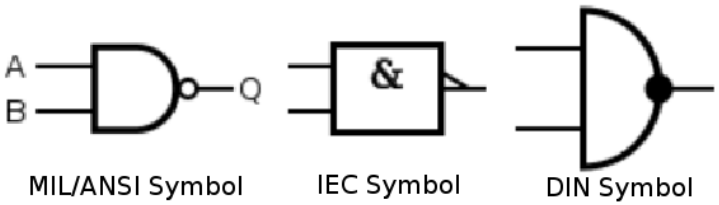
MUX symbol



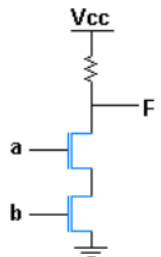
gate level MUX

Logical Unit

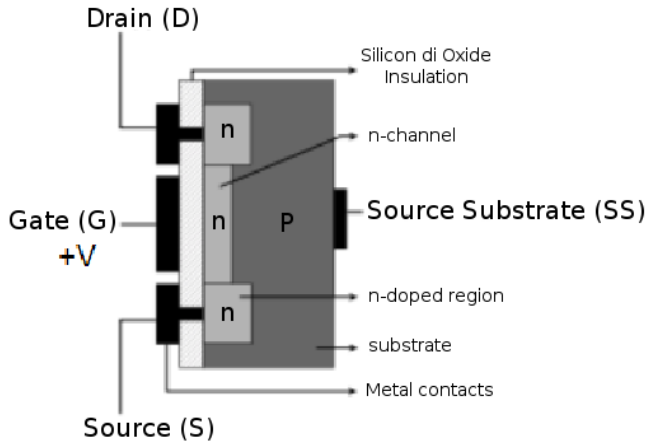




NAND gate symbols

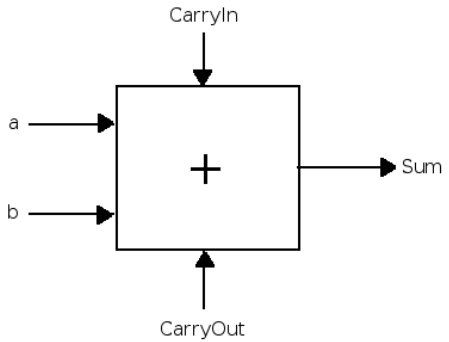


Transistor level NAND gate



NP regions of a MOS transistor.

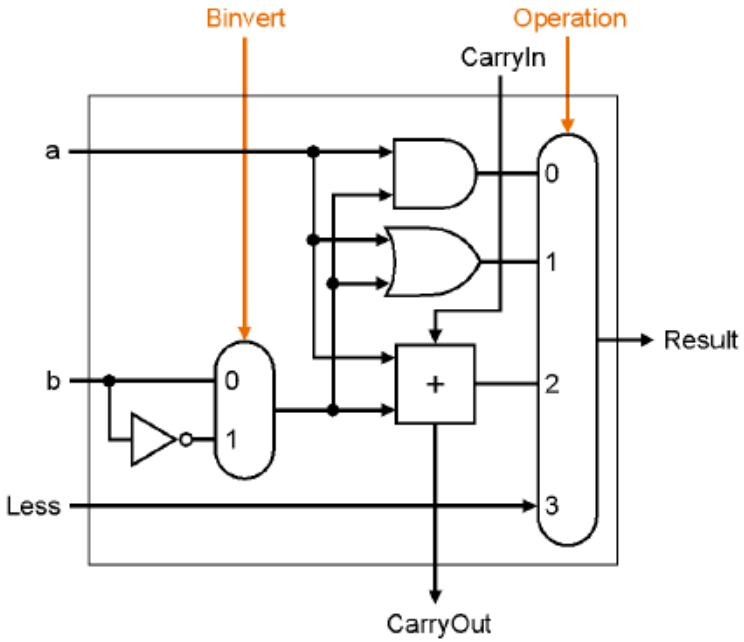
One-Bit Full Adder



Inputs			Outputs		Comments
a	b	Carry In (c_i)	Carry Out (c_o)	Sum (s)	(base 2)
0	0	0	0	0	$0+0+0=00$
0	0	1	0	1	$0+0+1=01$
0	1	0	0	1	$0+1+0=01$
0	1	1	1	0	$0+1+1=10$
1	0	0	0	1	$1+0+0=01$
1	0	1	1	0	$1+0+1=10$
1	1	0	1	0	$1+1+0=10$
1	1	1	1	1	$1+1+1=11$

One-Bit ALU

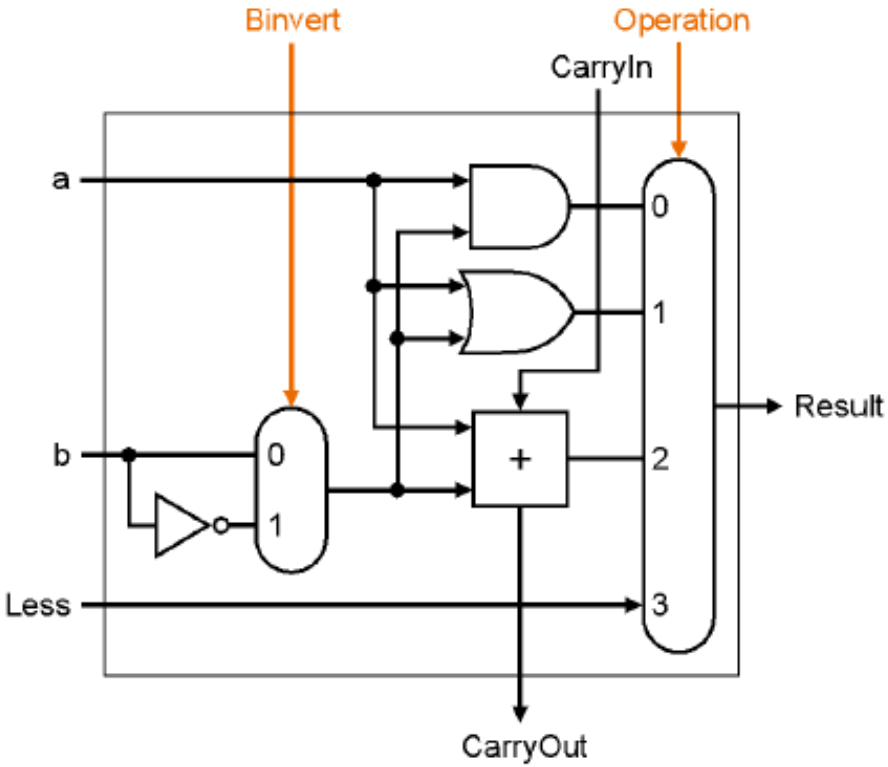
One-bit ALU for any of 31 LSBs



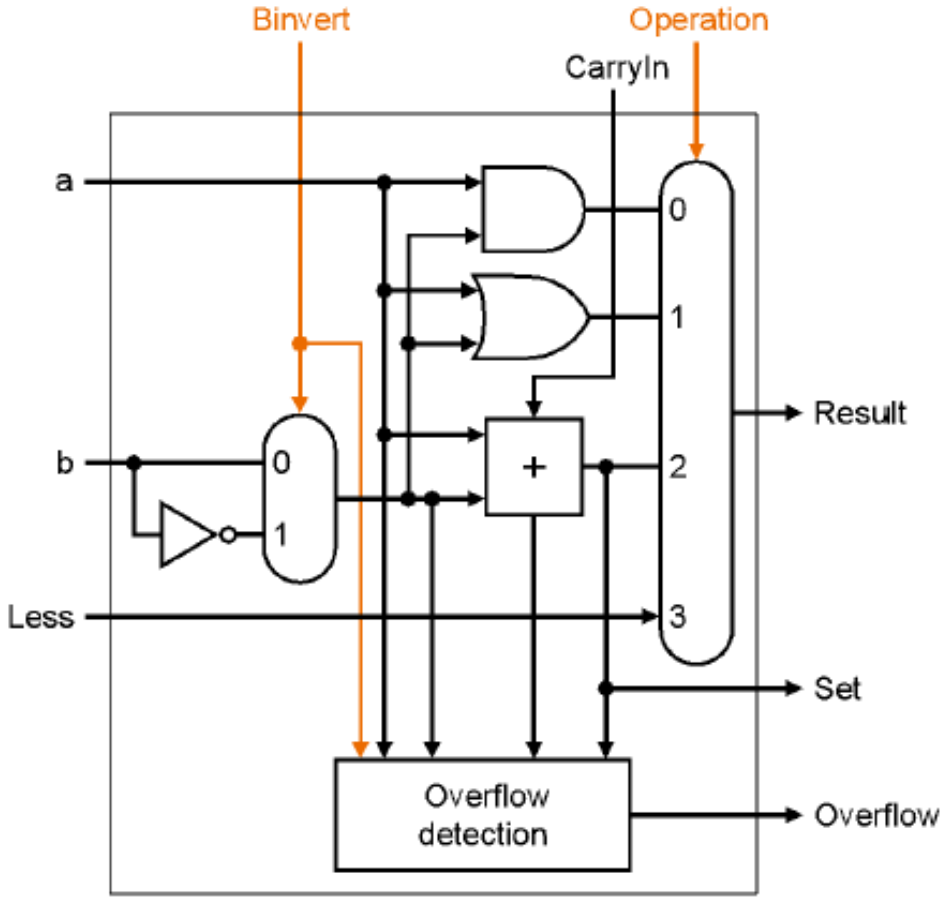
- x AND y: operation = [00], Binv = 0, a = x, b = y
- x OR y: operation = [01], Binv = 0, a = x, b = y
- NOT x: operation = [01], Binv = 1, a = 0, b = x

A 32-Bit ALU

One-bit ALU for any of 31 LSBs

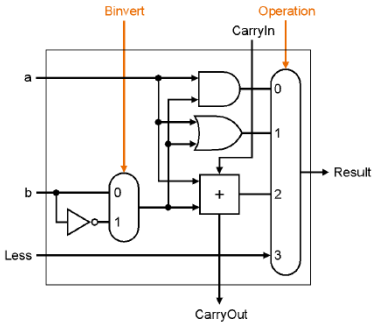


One-bit ALU for the MSB

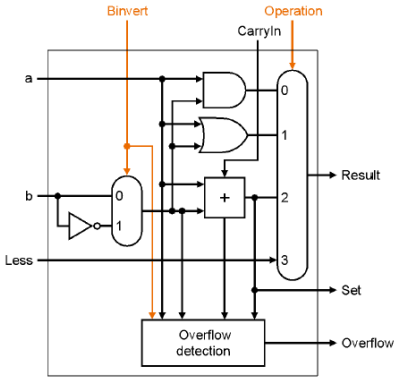


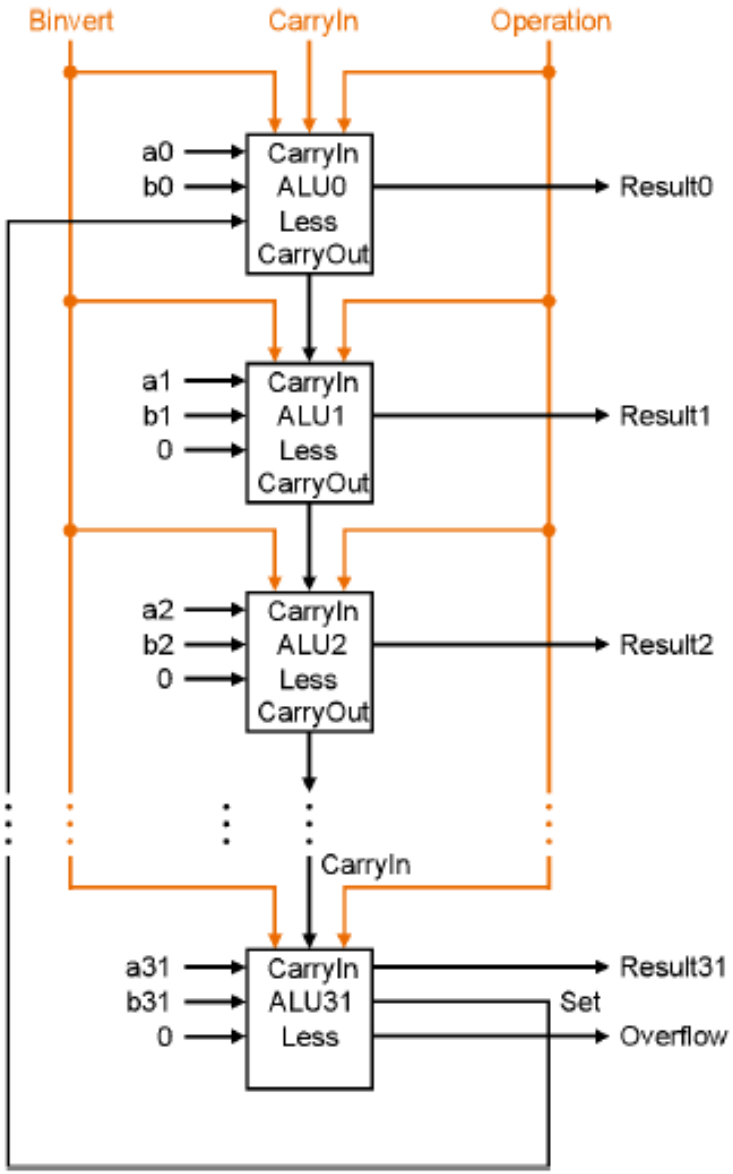
A 32-Bit ALU

31 LSBs



MSB

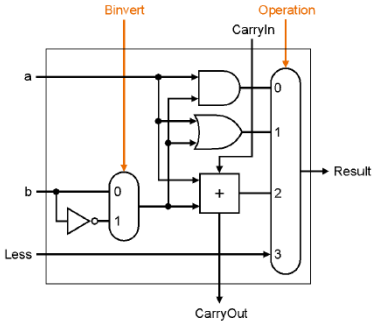




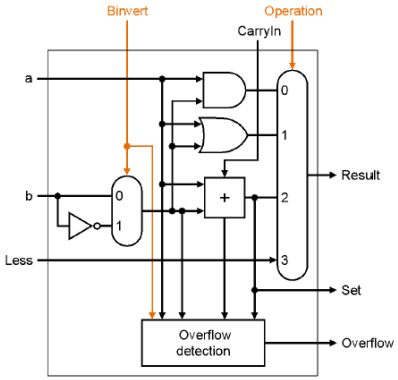
- x + y:
  operation = [10],
  Binv = 0, CarryIn[0] = 0
  a[i] = x[i], b[i] = y[i]

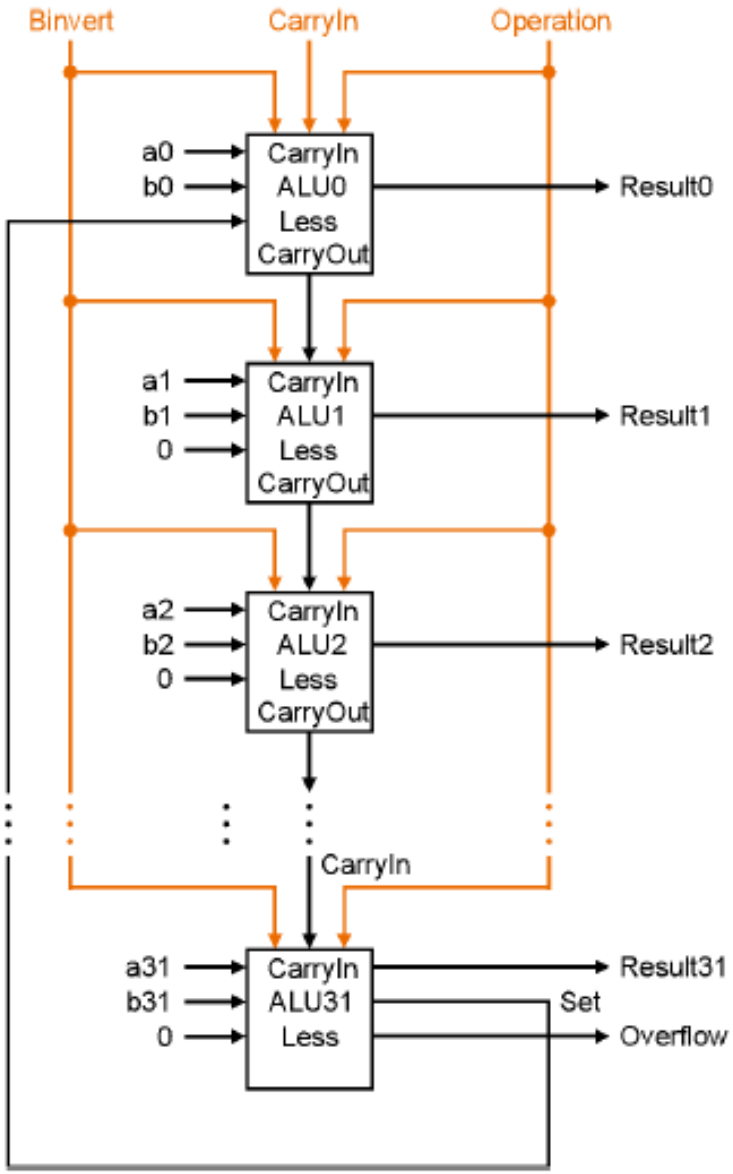
A 32-Bit ALU

31 LSBs



MSB

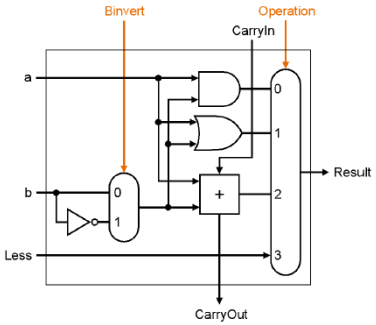




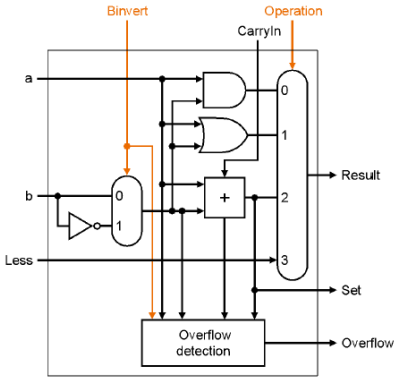
- x + y:
  operation = [10],
  Binv = 0, CarryIn[0] = 0
  a[i] = x[i], b[i] = y[i]
- x - y $= x + (-y)$:
  operation = [10],
  Binv = 1, CarryIn[0] = 1
  a[i] = x[i], b[i] = y[i]

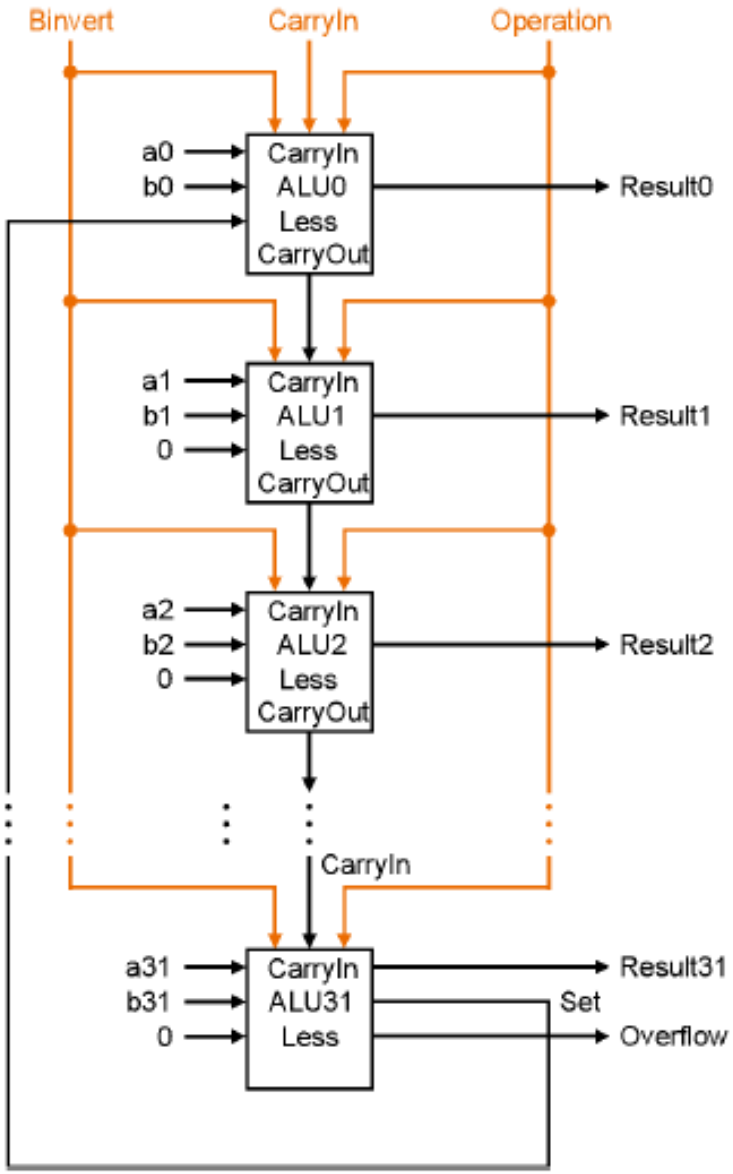
A 32-Bit ALU: Set on Less Than

31 LSBs



MSB





- slt rd, rs, rt

  (1) Do subtraction: $rs - rt$

  (2) If

  $rs < rt \Rightarrow (rs - rt) < 0$

  $\Rightarrow$ msb = 1

  (3) Let rd = [000...01] when msb = 1,

  otherwise rd = [000...00]

- slt rd, rs, rt:

  a[i] = rs[i], b[i] = rt[i]

  (1) Do subtraction

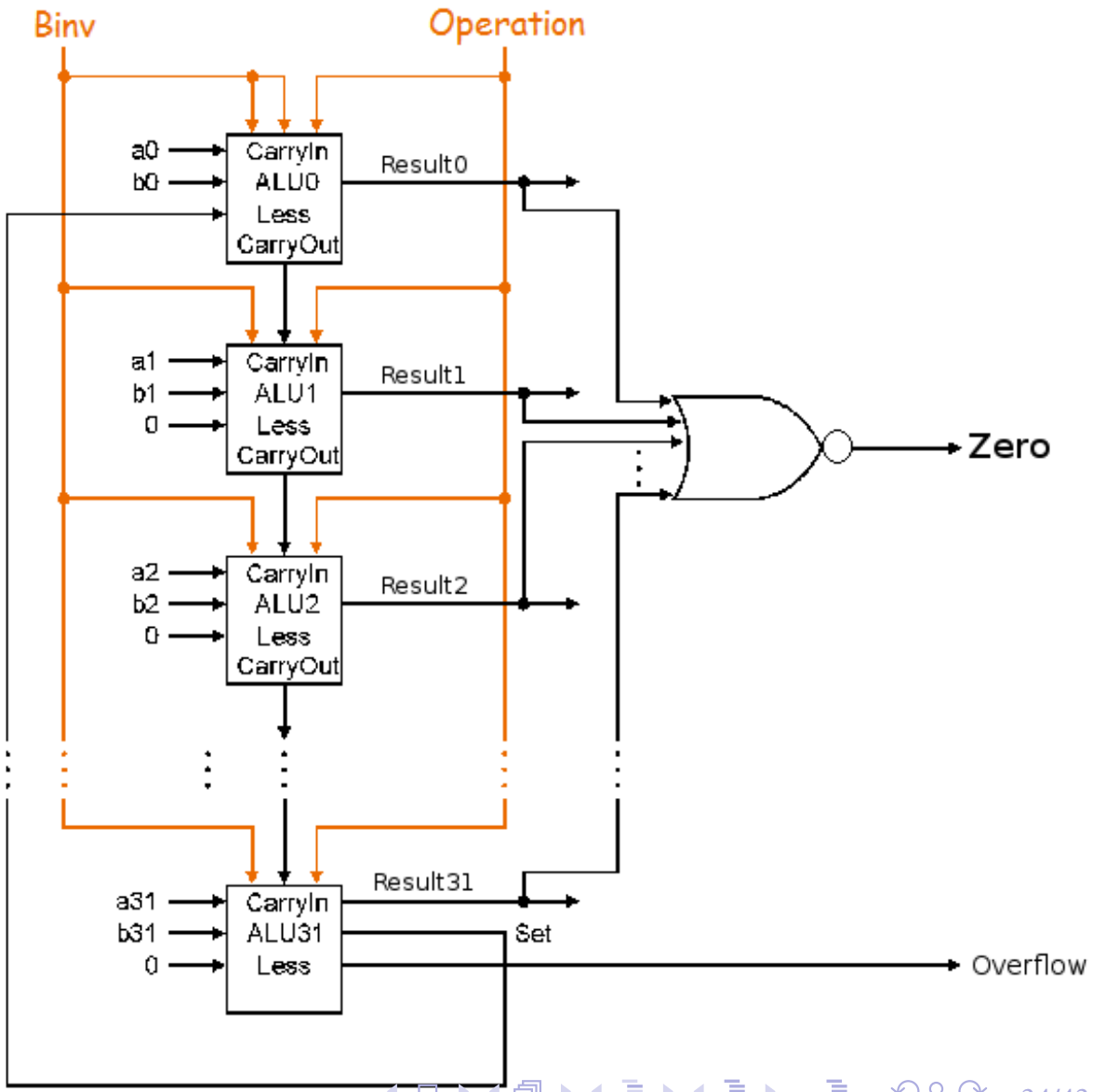
  Binv = 1, CarryIn[0] = 1

  (2-3) Set rd according to subtraction's msb

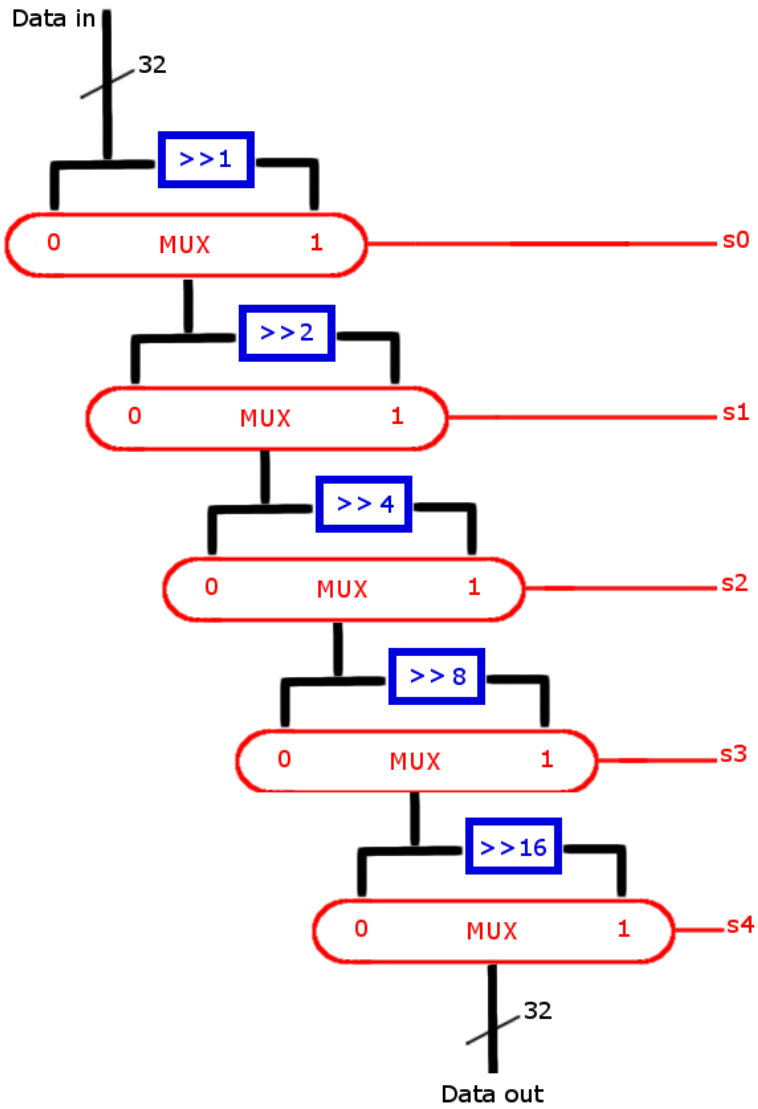
  operation = [11]

A 32-Bit ALU: Test on Equality

- Instructions, e.g.,
  beq rs, rt, offset, need support for test on equality
  (1) Do subtraction: $rs - rt$
  (2) If
  $rs = rt \Rightarrow (rs - rt) = 0$
  $\Rightarrow$ all bits $= 0$
- beq rs, rt, offset
  a[i] = rs[i], b[i] = rt[i]
  (1) Do subtraction
  Binv = 1, CarryIn[0] = 1
  (2) Check the result (with provided supporting hardware)
  operation = [10]

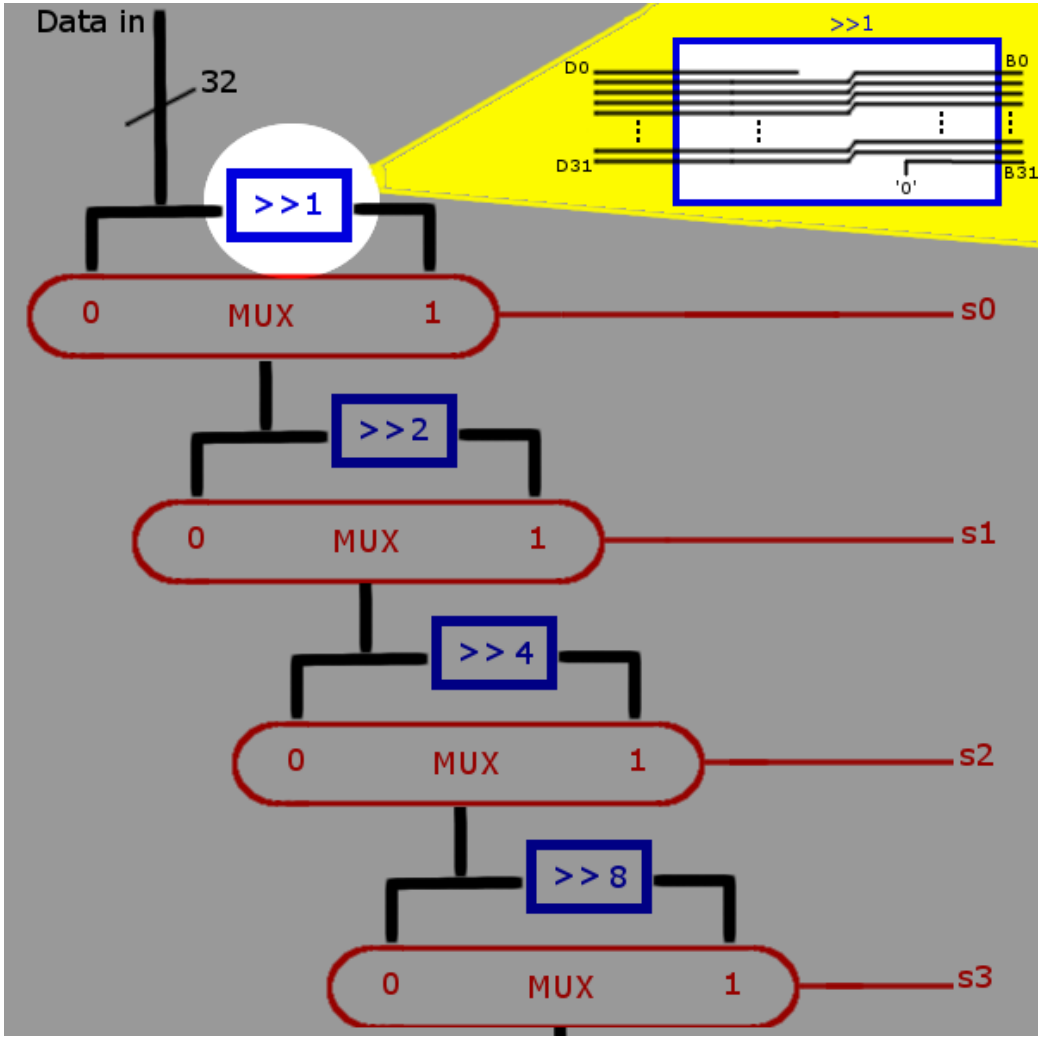


Barrel Shifter



- s = 0b00000 $\Rightarrow$ no shift
- s = 0b00001 $\Rightarrow$ shift 1 bit
- s = 0b00010 $\Rightarrow$ shift 2 bits
- s = 0b00011 $\Rightarrow$ shift 3 bits
- ...
- s = 0b11111 $\Rightarrow$ shift 31 bits

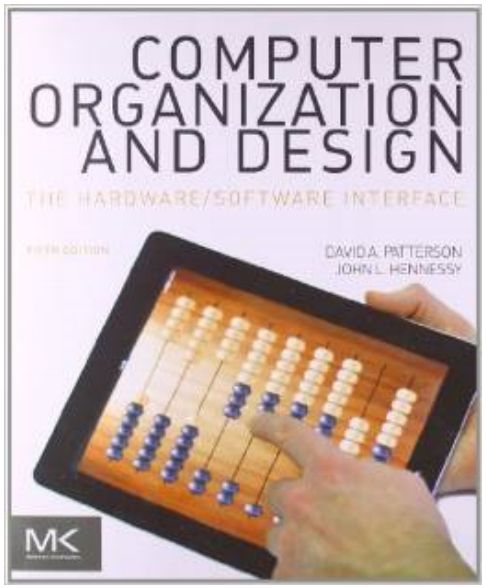
Shift Circuit



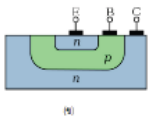
- Shift (or rotate) is rewiring. E.g., shift right 1 bit (>> 1) $\equiv$ rewire input bit 1 to output bit 0, input bit 2 to output bit 1, and so on. Then, fill '0' in as appropriate.

Closing note

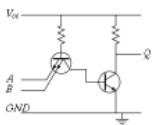
- อ่าน P&H5e, 3.1-3.2.
- อ่าน บทที่ 3 เอกสารประกอบรายวิชา 188 322 สถาปัตยกรรมคอมพิวเตอร์
  ธัชพงศ์ กตัญญูกุล, April 25, 2014.

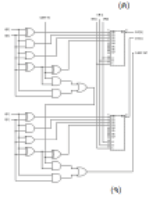












MIPS Architecture on Exception 1

- Check out https://www.mips.com/products/architectures/mips32/.
- MIPS Architecture For Programmers Vol. III:
  MIPS32/microMIPS32 Privileged Resource Architecture
  (#MD00090, Rev. 6.02, July 10, 2015)
  - "Unless the EXL bit in the Status register is already a 1, the processor writes the EPC register when an exception occurs." (pp. 216)
  - "The Exception Program Counter (EPC) is a read/write register that contains the address at which processing resumes after an exception has been serviced." (pp. 216)

MIPS Architecture on Exception 2

- "For synchronous (precise) exceptions, EPC contains either:
  • the virtual address of the instruction that was the direct cause of the exception, or
  • the virtual address of the immediately preceding branch or jump instruction, when the exception causing instruction is in a branch delay slot, and the Branch Delay bit in the Cause register is set."
- "For asynchronous (imprecise) exceptions, EPC contains the address of the instruction at which to resume execution. The processor reads the EPC register as the result of execution of the ERET instruction."
- synchronous or asynchronous exception is determined by the cause of exception. (Table 6.6, p. 83)

MIPS Architecture on Exception 3

Table 6.6 Priority of Exceptions

Exception	Description	Type
Reset	The Cold Reset signal was asserted to the processor	Asynchronous Reset
Soft Reset	The Reset signal was asserted to the processor	
Debug Single Step	An EJTAG Single Step occurred. Prioritized above other exceptions, including asynchronous exceptions, so that one can single-step into interrupt (or other asynchronous) handlers.	Synchronous Debug
Debug Interrupt	An EJTAG interrupt (EjtagBrk or DINT) was asserted.	Asynchronous Debug
Imprecise Debug Data Break	An imprecise EJTAG data break condition was asserted.	
Nonmaskable Interrupt (NMI)	The NMI signal was asserted to the processor.	Asynchronous
Machine Check	An internal inconsistency was detected by the processor.	
Interrupt	An enabled interrupt occurred.	
Deferred Watch	A watch exception, deferred because *EXL* was one when the exception was detected, was asserted after *EXL* went to zero.	
Debug Instruction Break	An EJTAG instruction break condition was asserted. Prioritized above instruction fetch exceptions to allow break on illegal instruction addresses.	Synchronous Debug

MIPS Architecture on Exception 4

Address Error - Instruction fetch	A non-word-aligned address was loaded into PC.	
TLB Refill - Instruction fetch	A TLB miss occurred on an instruction fetch.	
TLB Invalid - Instruction fetch	The valid bit was zero in the TLB entry mapping the address referenced by an instruction fetch.	
TLB Execute-Inhibit	An instruction fetch matched a valid TLB entry which had the XI bit set.	
Cache Error - Instruction fetch	A cache error occurred on an instruction fetch.	
Bus Error - Instruction fetch	A bus error occurred on an instruction fetch.	
SDBBP	An EJTAG SDBBP instruction was executed.	Synchronous Debug
Instruction Validity Exceptions	An instruction could not be completed because it was not allowed access to the required resources, or was illegal: Coprocessor Unusable, Reserved Instruction. If both exceptions occur on the same instruction, the Coprocessor Unusable Exception takes priority over the Reserved Instruction Exception.	Synchronous
Execution Exception	An instruction-based exception occurred: Integer overflow, trap, system call, breakpoint, floating-point, coprocessor 2 exception.	

MIPS Architecture on Exception 5

Another word,

- Exception from Interrupt (Asynchronous mode):
  $\Rightarrow$ EPC = (PC + 4) or pointing to the next instruction
- Exception from Overflow (Execution/Synchronous mode):
  $\Rightarrow$ EPC = PC or pointing to the instruction causing the exception

MIPS Architecture on Exception 6

MIPS Architecture for Programmers Vol II-A: The MIPS32 Instruction Set Manual
(MD00086, rev 6.06, Dec 15, 2016)

- ERET (p. 177): See details in MD00086.
- MFC0 (p. 266): GPR[rt] $\leftarrow$ CPR[0,rd,sel]
- MTC0 (p. 290): CPR[0, rd, sel] $\leftarrow$ GPR[rt]